



FlyAway

Giovanna Lani Ariño

Objetivo principal

FlyAway es una aplicación web colaborativa que permite a los usuarios planificar y organizar viajes de manera conjunta.

Los usuarios pueden crear y gestionar itinerarios, añadir destinos y actividades diarias, invitar a otros participantes, registrar y dividir gastos, y consultar información externa como el clima o lugares turísticos.



Funcionalidades

- **Gestión de usuarios:**
 - Registro y login (local y Google)
 - Perfil de usuario
 - Gestión de amigos y solicitudes
- **Gestión de viajes:**
 - Crear, actualizar y eliminar viajes
 - Añadir participantes y gestión de roles
 - Itinerario diario con actividades, imágenes y gastos
- **Datos externos:**
 - Búsqueda de lugares
 - Pronóstico Clima
 - Recomendaciones turísticas
 - Conversión de moneda



API

API Gateway

- **Puerto:** 3030
- Punto de entrada único del sistema
- Centraliza autenticación y routing

Frontend

- **Puerto:** 5173
- Interfaz de usuario
- Consumo del API Gateway

API

Users Trips Service

- **Puerto:** 3001
- Gestión de usuarios y autenticación
- Gestión de viajes, participantes, itinerarios y gastos

POST /api/auth/register	Registro de usuario
POST /api/trips	Creación de viaje
POST /api/expenses	Registro de gasto
GET /api/itinerary/trips/:tripId/itinerary	Itinerario del viaje
...	

API

External Data Service

- Puerto: 3002
- Consumo y cacheo de datos externos

GET /api/external/places/search?q=Paris	Búsqueda de lugares
POST /api/external/places/select	Selección de destino
GET /api/external/weather/:placeId?date=YYYY-MM-DD	Pronóstico del clima
GET /api/external/recommendations/:placeId	Recomendaciones turísticas
GET /api/external/currency/convert?from=EUR&to=USD	Conversión de moneda

Modelo de datos

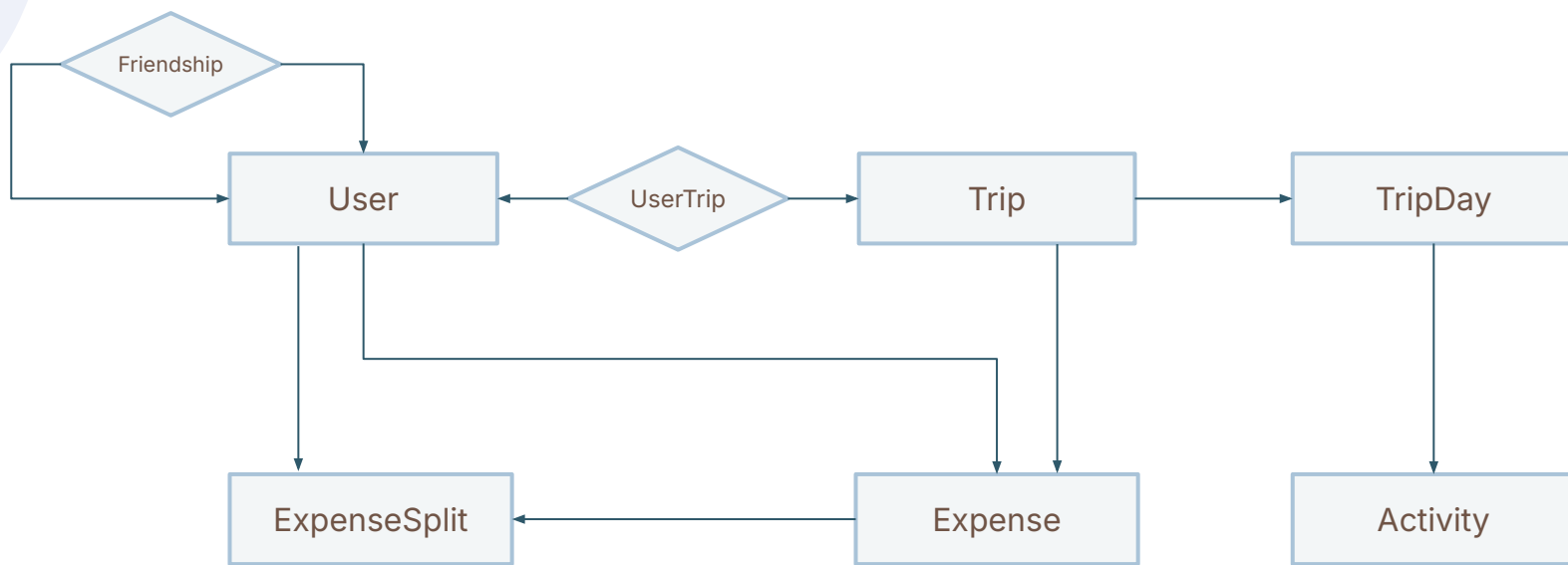
MySQL



- **User:** id, name, email, password, avatarUrl, bio, provider
- **Friendship:** userId, friendId, status
- **Trip:** id, name, description, startDate, imageUrl, isPublic
- **UserTrip:** userId, tripId, role
- **TripDay:** id, tripId, date, destinationPlaceId, destinationName
- **Activity:** id, tripDayId, name, place, startTime, endTime, price, description, images
- **Expense:** id, name, amount, currency, paidByUserId, tripId
- **ExpenseSplit:** expenseId, userId, amount, isSettled



Modelo de datos

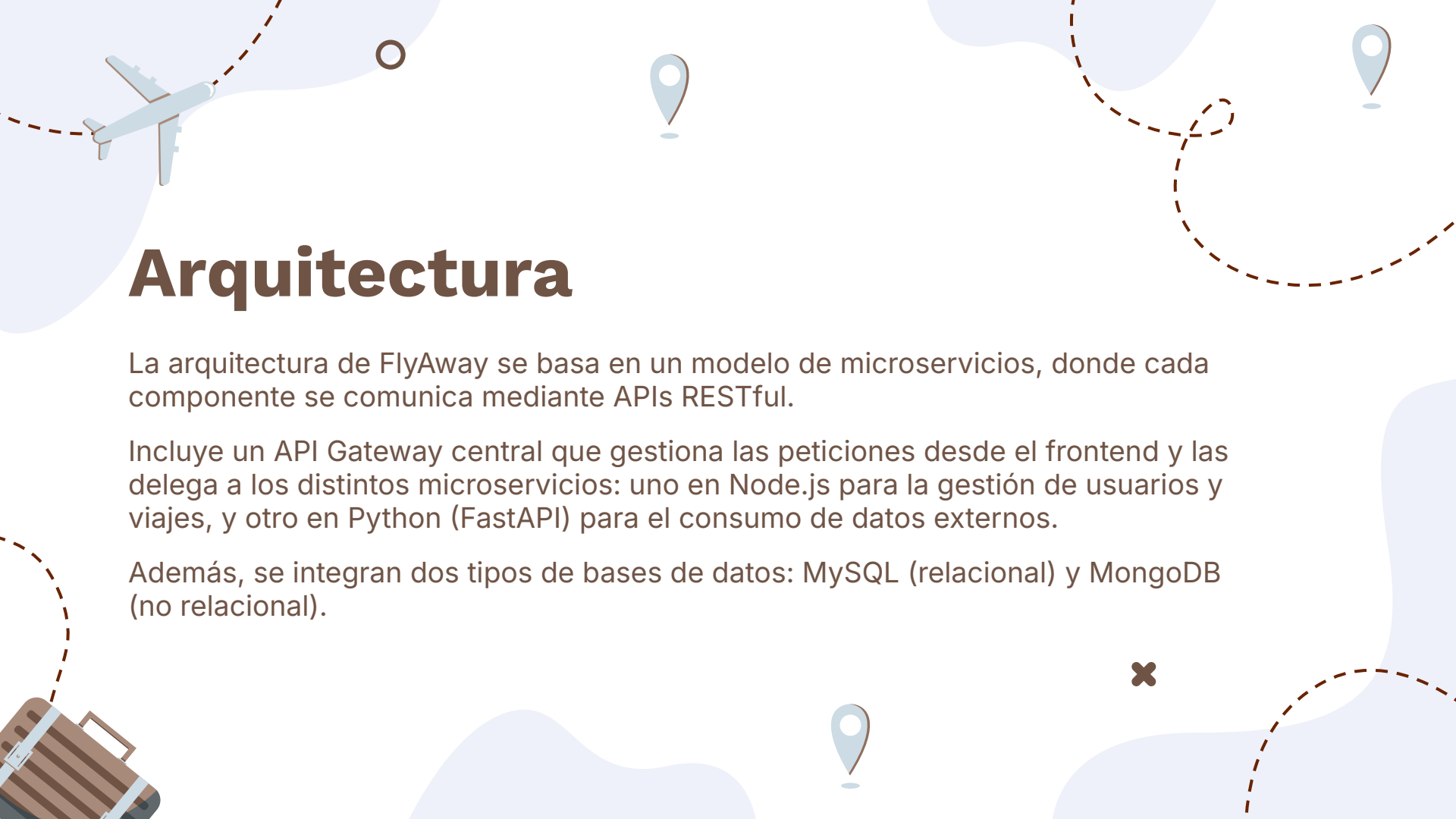


Modelo de datos

MongoDB



- **Place:** id, name, lat, lon, external_id
- **Weather:** place_id, date, data, expires_at
- **Recommendations:** place_id, radius, limit, items, expires_at



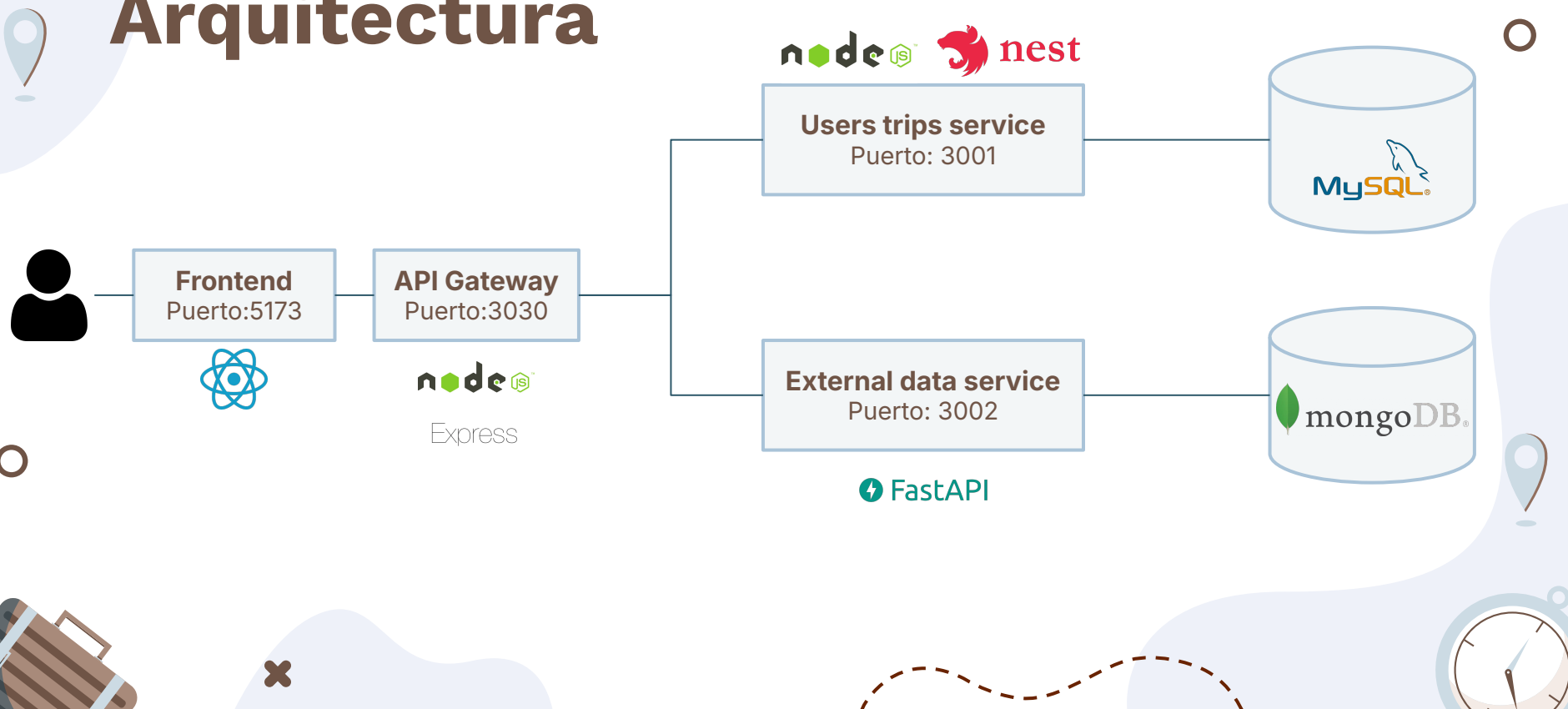
Arquitectura

La arquitectura de FlyAway se basa en un modelo de microservicios, donde cada componente se comunica mediante APIs RESTful.

Incluye un API Gateway central que gestiona las peticiones desde el frontend y las delega a los distintos microservicios: uno en Node.js para la gestión de usuarios y viajes, y otro en Python (FastAPI) para el consumo de datos externos.

Además, se integran dos tipos de bases de datos: MySQL (relacional) y MongoDB (no relacional).

Arquitectura



Tecnologías

	Tecnología	Descripción
Frontend	React + HTML5	Interfaz SPA
API Gateway	Node.js + Express	Punto central de entrada y enrutamiento de peticiones
Microservicio Viajes/Usuarios	Node.js + Nest	CRUD de usuarios, viajes, itinerarios y gastos
Microservicio datos externos	Python + FastAPI	Consulta datos externos, guarda caché
APIs RESTful	OpenAPI 3.0	Documentación y especificación de endpoints
BBDD relacional	MySQL	Datos de usuarios, viajes, actividades, gastos
BBDD no relacional	MongoDB	Almacenamiento caché de consultas
Infraestructura	Docker	Despliegue y aislamiento de microservicios



Datos externos

	API	Descripción
Lugares	OpenStreetMap	Buscar y seleccionar lugares turísticos
Clima	Open-Meteo	Pronóstico del tiempo por lugar y fecha
Recomendaciones	Overpass API	Sugiere lugares cercanos al destino, con límite y radio configurable
Moneda	Frankfurter API	Conversión de moneda



SPA (Single Page Application)

La aplicación es SPA usando React: toda la navegación se hace sin recargar la página.

PWA (Progressive Web App)

Implementada como PWA:

- Service Worker cachea archivos estáticos y algunas respuestas API.
- Permite funcionamiento offline parcial.
- Se puede agregar a la pantalla de inicio

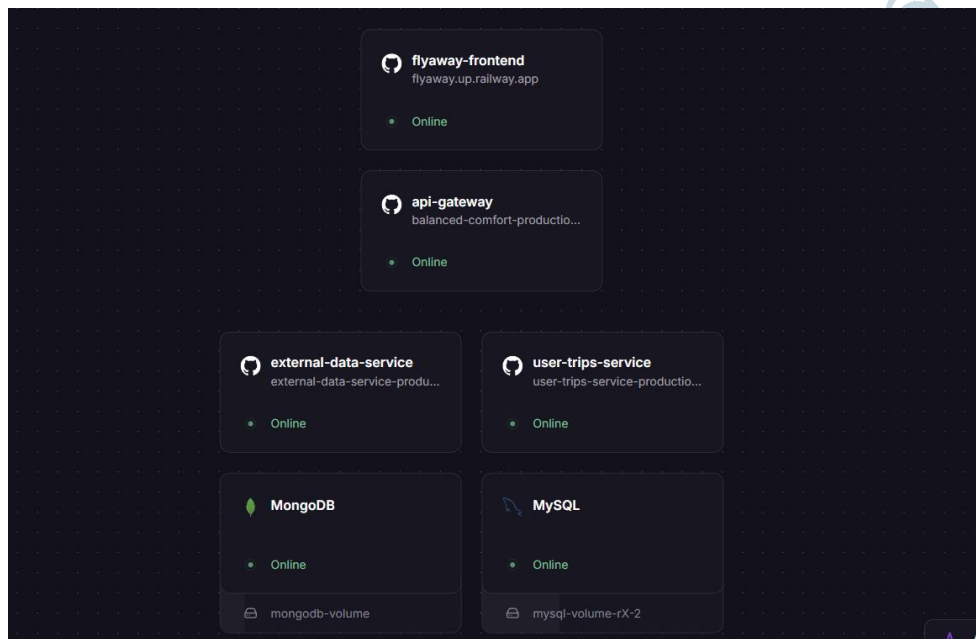


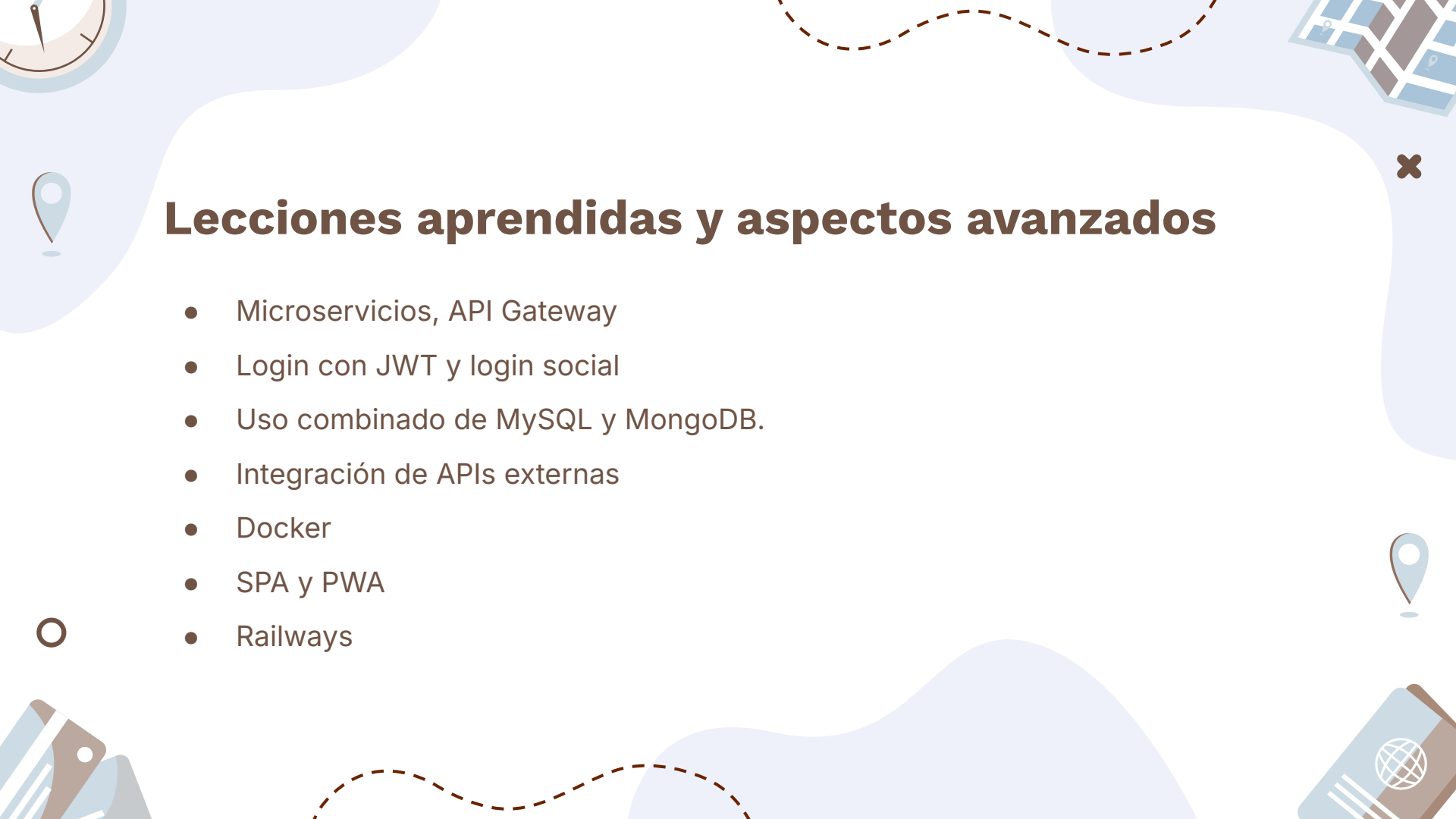
Cliente HTML5 y JavaScript

- Interfaz construida con HTML5 + CSS + React + JS moderno (ES6+).
- No se usan plantillas del servidor; toda la lógica de UI está en el cliente.
- Consumo de APIs RESTful para datos (usuarios, viajes, itinerarios, gastos, datos externos).
- Librerías principales:
 - React Router para rutas internas
 - Bootstrap + Reactstrap para diseño
 - Fetch API para llamadas a microservicios

Subida a la nube con Railways

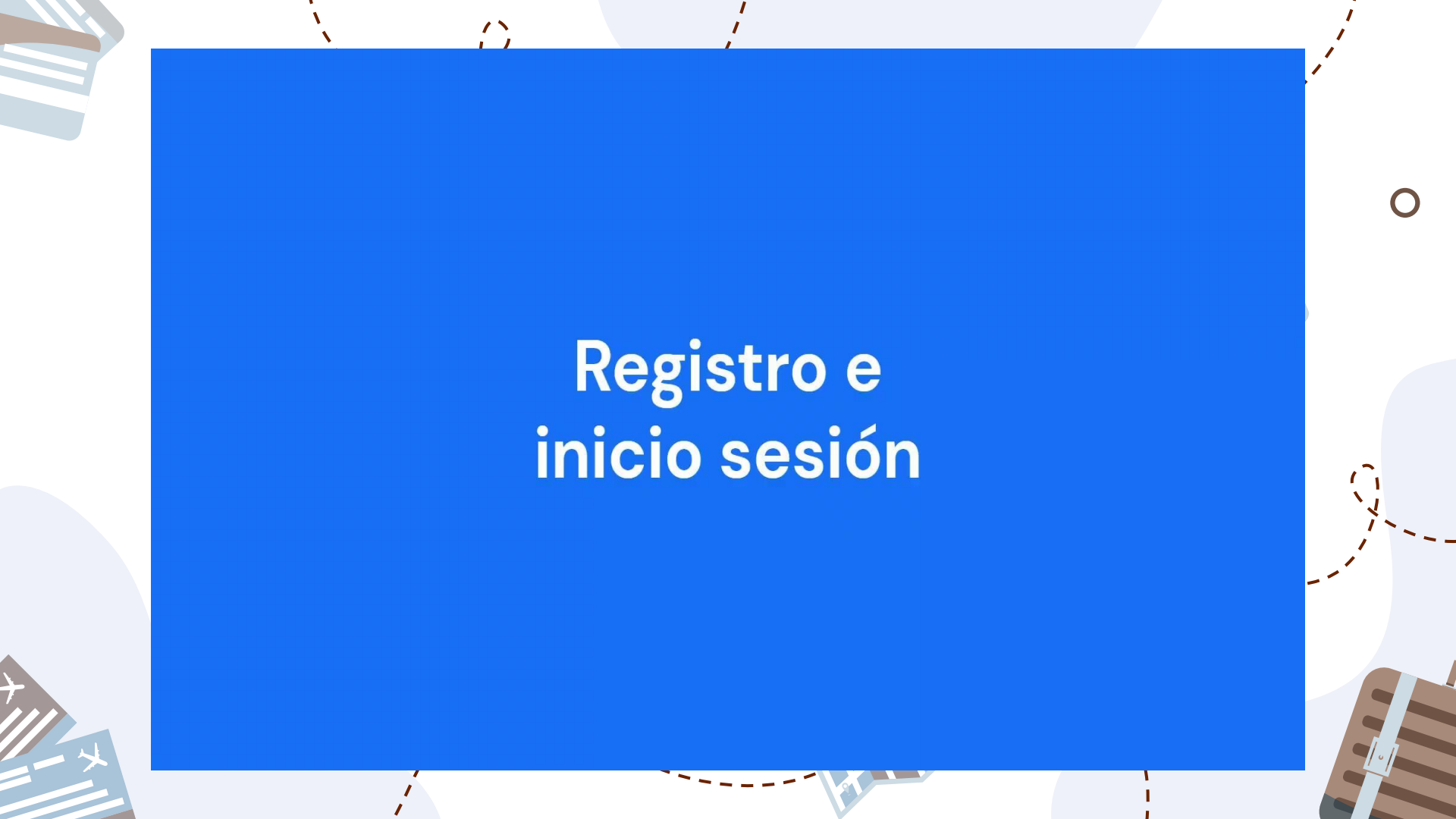
<https://flyaway.up.railway.app/>





Lecciones aprendidas y aspectos avanzados

- Microservicios, API Gateway
- Login con JWT y login social
- Uso combinado de MySQL y MongoDB.
- Integración de APIs externas
- Docker
- SPA y PWA
- Railways



Registro e inicio sesión



Conclusiones

Se ha desarrollado una aplicación completa y funcional, cubriendo frontend, backend y arquitectura.

La separación en microservicios y el uso de un API Gateway ha facilitado la escalabilidad y el mantenimiento.

El uso de una SPA con React mejora la experiencia de usuario y la fluidez de la aplicación.

La incorporación de PWA demuestra cómo una aplicación web puede ofrecer características similares a una app nativa.

Se ha aprendido a aplicar conceptos reales de desarrollo web moderno de forma integrada.

